claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-ad8ef9675431552fe.jsonl`
- SHA-256: `0c329e7997a6a689e200a757a047f633378837bd195431c3dd2cc86423c51e12`
- Source modified: `2026-07-02T03:02:02+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T02:59:43.306Z)

```
You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.
A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:

{
  "title": "Fetched-input scratch copies are never reclaimed anywhere (process jobs, compile jobs, /api/assets, cloud_run marker reads) ? 'OS temp reclamation' premise is false on Windows",
  "category": "tech-debt",
  "severity": "P2",
  "file": "backend/storage/__init__.py",
  "line": 89,
  "evidence": "storage/__init__.py:80-90: acquire_local_input docstring says the scratch copy 'is left under the system temp dir ... and relies on OS temp reclamation'. Call sites that leak one full-video copy per run: _execute_processing (main.py:1168-1171), _execute_compile (main.py:1683-1686), create_asset (main.py:791-799), get_source (main.py:646). cloud_run.py:219-222 additionally leaks a cr-read-* mkdtemp per marker/report read. No janitor exists (grep for rally-input-/cleanup found none); engine issue #301 covers only the upload_dir retention sweep, not temp scratch.",
  "why_it_matters": "Slow-burn disk exhaustion on exactly the appliance the product ships as; also inflates the free-space guard's false rejections.",
  "proposed_fix": "Wrap remote-input acquisition in a per-job scope that deletes the scratch dir in a finally (worker already owns the job lifecycle), or route scratch under output_base(cfg)/tmp with an age-based sweep at startup; make cloud_run._read_json use TemporaryDirectory.",
  "effort": "M",
  "tracked_in": ""
}

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).
VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?
HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code,
```

say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T02:59:50.705Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13
14      from __future__ import annotations
15
16      import os
17      import tempfile
18      from typing import Any, Dict, Iterator, Optional
19
20      from backend.storage.base import StorageBackend
21      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
22
23      __all__ = [
24          "StorageBackend",
25          "StorageRef",
26          "StorageStat",
27          "parse_uri",
28          "resolve_input_uri",
29          "get_storage",
30          "fetch",
31          "put",
32          "list_uris",
33          "acquire_local_input",
34          "input_is_local",
35          "resolve_input_ref",
36      ]
37
38
39      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
40          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
41          if storage_config is None:
42              return {}
43          if hasattr(storage_config, "model_dump"):
44              return storage_config.model_dump()  # type: ignore[no-any-return]
45          return dict(storage_config)
46
47
48      def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
49          """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
50          config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
51          callers at the API/CLI boundary catch that and return a clean client error (never a
500).
52
53          Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
```

```
        ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
        sc = _storage_dict(storage_config)
        uri = resolve_input_uri(
            raw,
            input_backend=sc.get("input_backend", "local"),
            input_root=sc.get("input_root", ""),
        )
        return parse_uri(uri)


    def input_is_local(raw: str, storage_config: Any = None) -> bool:
        """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
        plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
        that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
        ``resolve_input_ref``)."""
        return resolve_input_ref(raw, storage_config).backend == "local"


    def acquire_local_input(
        raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
    ) -> str:
        """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.

        Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
        byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
        non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
        path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
        regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
        worker's ``cmd_infer`` already ships."""
        sc = _storage_dict(storage_config)
        ref = resolve_input_ref(raw, sc)
        if ref.backend == "local":
            return (
                ref.key
            )  # identity ? no temp, no copy, byte-identical to passing the path directly
        scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
        return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)


    def _backend_class(name: str):
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage

            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage

            return S3Storage
        if name == "gcs":
            from backend.storage.gcs import GCSStorage

            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage

```

```
            return GDriveStorage
        if name == "gdrive-api":
            from backend.storage.gdrive_api import GDriveApiStorage

            return GDriveApiStorage
        raise ValueError(
            f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
        )


    def get_storage(
        backend: Optional[str] = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        **overrides,
    ) -> StorageBackend:
        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
        the implementation; its per-backend settings come from ``config[backend]`` merged
with
        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
        cfg = config or {}
        name = backend or cfg.get("backend", "local")
        # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
        # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
        # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
        settings: Dict[str, Any] = dict(
            cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
        )
        settings.update(overrides)
        return _backend_class(name)(**settings)


    def fetch(
        uri: str,
        dst: "str | os.PathLike[str] | None" = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        overwrite: bool = False,
    ) -> str:
        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
        (no copy); for s3/gcs it downloads to ``dst``."""
        ref = parse_uri(uri)
        return get_storage(ref.backend, config=config).fetch_to_local(
            ref, dst, overwrite=overwrite
        )


    def put(
        local_path: "str | os.PathLike[str]",
        uri: str,
        *,
        config: Optional[Dict[str, Any]] = None,
        content_type: Optional[str] = None,
    ) -> str:
        """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
        ref = parse_uri(uri)
        get_storage(ref.backend, config=config).put(
            local_path, ref, content_type=content_type
        )
        return uri
```

```
169
170
171     def list_uris(
172         prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
173     ) -> Iterator[str]:
174         """Yield the URI of every object under ``prefix_uri``."""
175         ref = parse_uri(prefix_uri)
176         for r in get_storage(ref.backend, config=config).list(ref):
177             yield r.uri
178
```

3. user (2026-07-02T02:59:52.522Z)

```
b6214e5 chore: ruff format the whole repo (259 files) (#387)
819e395 feat(serving): M12 ? gdrive-api:// OAuth/API StorageBackend (D14 Northstar) (#338)
e111add feat(serving): M2 ? URI-aware input (input_backend/input_root, local=identity) (#280)
38c2d0f feat(storage): gdrive:// reads a locally MOUNTED Google Drive (no API) (#258)
6878d45 feat(storage): pluggable StorageBackend ? local + S3/GCS first-class buckets (#249)
---
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
```

4. user (2026-07-02T03:00:02.579Z)

```
badminton-highlight-indexer\tests\test_api_cloud_dispatch.py:114:        m.storage,
"acquire_local_input", lambda path, scfg: str(fake_local)
badminton-highlight-indexer\tests\test_api_cloud_dispatch.py:381:        m.storage,
"acquire_local_input", lambda path, scfg: str(fake_local)
badminton-highlight-indexer\tests\test_api_cloud_dispatch.py:556:        m.storage,
"acquire_local_input", lambda path, scfg: str(fake_local)
badminton-highlight-indexer\backend\config\models.py:378:    # scratch dir before processing
(see ``backend.storage.acquire_local_input``).
badminton-highlight-indexer\docs\PACKAGING_PLAN.md:154:| **CUJ-2** In-UI `gs://` bucket ingest |
DONE | `acquire_local_input` + gcs backend already accept `gs://`; preserved when the user opts
into a cloud storage backend (+ ADC). |
badminton-highlight-indexer\tests\test_input_source.py:2:acquire_local_input.
badminton-highlight-indexer\tests\test_input_source.py:19:from backend.storage import
acquire_local_input, input_is_local, resolve_input_ref
badminton-highlight-indexer\tests\test_input_source.py:124:#
------------------------------------------------------------- acquire_local_input
badminton-highlight-indexer\tests\test_input_source.py:125:def
test_acquire_local_input_local_identity(tmp_path):
badminton-highlight-indexer\tests\test_input_source.py:129:    assert
acquire_local_input(str(p)) == str(p)
badminton-highlight-indexer\tests\test_input_source.py:132:def
test_acquire_local_input_relative_local_identity():
badminton-highlight-indexer\tests\test_input_source.py:133:    assert
acquire_local_input("rel/clip.mp4") == "rel/clip.mp4"
badminton-highlight-indexer\tests\test_input_source.py:149:def
test_acquire_local_input_remote_fetches_to_scratch(monkeypatch):
badminton-highlight-indexer\tests\test_input_source.py:152:    out = acquire_local_input(
badminton-highlight-indexer\tests\test_input_source.py:162:def
test_acquire_local_input_input_root_joins_then_fetches(monkeypatch):
badminton-highlight-indexer\tests\test_input_source.py:165:    acquire_local_input("x.mp4",
{"input_root": "gs://bucket/videos"})
badminton-highlight-indexer\tests\test_input_source.py:169:def
test_acquire_local_input_never_fetches_for_local(monkeypatch):
badminton-highlight-indexer\tests\test_input_source.py:174:    assert
acquire_local_input("/data/x.mp4") == "/data/x.mp4"
badminton-highlight-indexer\tests\test_input_source.py:176:
acquire_local_input("clip.mp4", {"input_backend": "local", "input_root": ""})
badminton-highlight-indexer\backend\main.py:646:        local =
```

```
storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage", None))
badminton-highlight-indexer\backend\main.py:791:            local_working =
storage.acquire_local_input(
badminton-highlight-indexer\backend\main.py:1168:    local_video = storage.acquire_local_input(
badminton-highlight-indexer\backend\main.py:1683:        local_src =
storage.acquire_local_input(
badminton-highlight-indexer\backend\main.py:2056:        local_video =
storage.acquire_local_input(args.video_path, storage_cfg)
badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\README.md:161:- 2026-06-21 ? **M2
landed** ([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280)):
`input_backend`/`input_root` + URI-aware main API/CLI input via
`resolve_input_uri`/`acquire_local_input` (local = identity passthrough, byte-for-byte
unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted
on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report
provenance. Suite 1105 green, cov 84.84%.
badminton-highlight-indexer\backend\storage\__init__.py:33:    "acquire_local_input",
badminton-highlight-indexer\backend\storage\__init__.py:72:def acquire_local_input(
```

5. user (2026-07-02T03:00:04.047Z)

```
badminton-highlight-indexer\backend\main.py:892:    with tempfile.TemporaryDirectory(prefix="cr-
ingest-") as td:
badminton-highlight-indexer\backend\main.py:1483:# sequential-part logic, and abort cleanup ? is
unchanged.
badminton-highlight-indexer\backend\main.py:1823:        deleted =
provider.cleanup_orphaned_files()
badminton-highlight-indexer\backend\providers\gemini.py:28:# are ever deleted by
cleanup_orphaned_files().
badminton-highlight-indexer\backend\providers\gemini.py:53:        """Best-effort cleanup of an
uploaded File (never raises)."""
badminton-highlight-indexer\backend\providers\gemini.py:59:    def cleanup_orphaned_files(self,
grace_sec: int = 3600) -> int:
badminton-highlight-indexer\backend\providers\gemini.py:62:        leaves the uploaded clip
behind forever ? the per-call cleanup only covers paths
badminton-highlight-indexer\backend\utils\annotation_proxy.py:47:def _cleanup(path: str) ->
None:
badminton-highlight-indexer\backend\utils\annotation_proxy.py:84:        _cleanup(tmp)
badminton-highlight-indexer\backend\utils\annotation_proxy.py:90:        _cleanup(tmp)
badminton-highlight-indexer\backend\utils\annotation_proxy.py:96:        _cleanup(tmp)
badminton-highlight-indexer\backend\storage\__init__.py:89:    scratch =
tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
badminton-highlight-indexer\backend\compute\cloud_run.py:220:
tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
badminton-highlight-indexer\backend\eval\golden_real_fixtures.py:393:
shutil.rmtree(str(tmp_parent), ignore_errors=True)
badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:419:        with
tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
badminton-highlight-indexer\backend\pipeline\detectors\base.py:162:            f"Free space
(e.g. python -m tools.cleanup_caches) and retry."
badminton-highlight-indexer\backend\pipeline\detectors\base.py:166:        """Best-effort
recursive delete of WSL paths (post-success cleanup; never raises)."""
badminton-highlight-indexer\backend\pipeline\detectors\base.py:174:
logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:129:        Returns
(video_to_upload, cleanup_temp_files) where cleanup_temp_files removes any
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:183:        def
cleanup_temp_files():
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:190:        return
video_to_upload, cleanup_temp_files
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:203:
cleanup_temp_files: Callable[[], None],
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:375:
cleanup_temp_files()
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:394:
cleanup_temp_files: Callable[[], None],
```

```
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:428:
cleanup_temp_files()
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:630:        # Optionally trim
the video first (debug feature); cleanup removes any temp file.
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:631:        video_to_upload,
cleanup_temp_files = self._maybe_trim_video(
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:655:        # video keeps its
own dir, so cleanup here never removes another run's chunks.
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:669:
cleanup_temp_files,
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:678:
cleanup_temp_files,
badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py:175:
"""Best-effort recursive delete of a native path (post-success cleanup; never raises)."""
badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py:177:
shutil.rmtree(path, ignore_errors=True)
```

6. user (2026-07-02T03:00:12.502Z)

```
630         rec = request.app.state.db.get_video(video_id)
631         if not rec or not rec.get("filepath"):
632             raise HTTPException(status_code=404, detail="Video not found.")
633         cfg = request.app.state.config
634
635         # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
636         try:
637             proxy_name = safe_output_filename(f"{video_id}.mp4")
638             proxy_path = resolve_under_root(os.path.join(_proxies_dir(cfg), proxy_name),
_proxies_dir(cfg))
639         except ValueError:
640             proxy_path = ""
641         if proxy_path and os.path.isfile(proxy_path):
642             return FileResponse(proxy_path, media_type="video/mp4")
643
644         # 2) Otherwise resolve + serve the source.
645         try:
646             local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage",
None))
647             is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage",
None))
648         except ValueError as e:  # unsupported scheme etc. ? a clean client error, never a
500
649             raise HTTPException(status_code=400, detail=str(e)) from e
650         except Exception as e:  # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
(not swallowed)
651             raise HTTPException(status_code=502, detail=f"Could not resolve the source
video: {e}") from e
652         if not os.path.isfile(local):
653             raise HTTPException(status_code=404, detail="Source video file not found.")
654
655         # 3) Warm a proxy in the background for next time (local source only).
656         if is_local:
657             threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()
658
659         media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
660         return FileResponse(local, media_type=media)
661
662
663     @app.post("/api/annotations")
664     def post_annotations(req: AnnotationsRequest, request: Request):
665         """Persist finished rally labels as the annotator CSV (P8), written verbatim to
666         ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5
``video_id`` ? so it joins
667         the video by content hash exactly as the training pipeline expects. Byte-compatible
```

```
with the VLC
668         plugin + browser extension. A later step (owner-gated) promotes labels into the
durable corpus.
669
670         Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is
validated as a bare
671         filename so an untrusted value can't escape the labels dir (arbitrary write)."""
672         try:
673             edge_auth.resolve_tenant(request.headers, request.app.state.config)
674         except edge_auth.EdgeAuthError as e:
```

7. user (2026-07-02T03:00:13.091Z)

```
770                 validate_input_path(req.uploaded_path, allowed_root=None)
771                 src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
772             except ValueError as e:
773                 raise HTTPException(status_code=400, detail=str(e)) from e
774             if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
775                 raise HTTPException(
776                     status_code=404,
777                     detail=f"uploaded file not found at '{req.uploaded_path}'",
778                 )
779             local_working = src_ref.key
780         else:
781             try:
782                 validate_input_path(req.source_uri, allowed_root=None)
783                 src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
784             except ValueError as e:
785                 raise HTTPException(status_code=400, detail=str(e)) from e
786             # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that
scratch (P2).
787             if src_ref.backend != "local":
788                 need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
789                 require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
790             try:
791                 local_working = storage.acquire_local_input(
792                     req.source_uri, getattr(cfg, "storage", None)
793                 )
794             except ValueError as e:
795                 raise HTTPException(status_code=400, detail=str(e)) from e
796             except Exception as e:  # noqa: BLE001 ? a fetch miss is a 502, surfaced not
swallowed
797                 raise HTTPException(
798                     status_code=502, detail=f"could not resolve source_uri: {e}"
799                 ) from e
800             if not os.path.isfile(local_working):
801                 raise HTTPException(
802                     status_code=404, detail="source_uri did not resolve to a readable file."
803                 )
804
805         video_id = compute_video_id(local_working)
806         ext = os.path.splitext(local_working)[1] or ".mp4"
807
808         # 2) Decide the durable location + copy only when needed.
809         copied = False
810         if req.source_uri:
811             stored_uri = req.source_uri  # already at rest in its medium ? no copy (D14b)
812             medium_backend = src_ref.backend
813         else:
814             try:
815                 medium_ref = storage.parse_uri(req.medium_uri)
816             except ValueError as e:
817                 raise HTTPException(
818                     status_code=400, detail=f"invalid medium_uri: {e}"
819                 ) from e
```

```
820             medium_backend = medium_ref.backend
821             if medium_ref.backend == "local":
822                 # "This computer" ? the upload already lives here; it IS the durable copy
(no duplicate).
823                 stored_uri = os.path.abspath(local_working)
824             else:
825                 dest = _asset_dest_uri(req.medium_uri, video_id, ext)
826                 # Free-space: BLOCK a local/Drive-mount destination (D11); a cloud bucket is
warn-only.
827                 local_dir = _medium_local_dir(medium_ref, storage_settings)
828                 if local_dir is not None:
829                     require_free_bytes(
```

8. user (2026-07-02T03:00:22.496Z)

```
829                     require_free_bytes(
830                         local_dir, os.path.getsize(local_working), stage="store"
831                     )
832                 else:
833                     logger.info(
834                         "[ASSETS] cloud medium %s ? capacity not blocked (cost/quota warn
only).",
835                         medium_ref.backend,
836                     )
837                 try:
838                     stored_uri = storage.put(local_working, dest, config=storage_settings)
839                     copied = True
840                 except ValueError as e:
841                     raise HTTPException(status_code=400, detail=str(e)) from e
842                 except Exception as e:  # noqa: BLE001 ? a put failure is surfaced, never
silently dropped
843                     logger.error(
844                         "[ASSETS] put failed video_id=%s dest=%s: %s",
845                         video_id,
846                         dest,
847                         e,
848                         exc_info=True,
849                     )
850                     raise HTTPException(
851                         status_code=502, detail=f"could not store into the medium: {e}"
852                     ) from e
853
854         # 3) Register the asset (keyed by MD5) so it appears in GET /api/videos ("my
videos").
855         if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
856             raise HTTPException(status_code=500, detail="failed to register the asset.")
857         logger.info(
858             "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
859             video_id,
860             medium_backend,
861             stored_uri,
862             copied,
863             owner_id,
864         )
865         return JSONResponse(
866             status_code=201,
867             content={
868                 "video_id": video_id,
869                 "medium": medium_backend,
870                 "stored_uri": stored_uri,
871                 "copied": copied,
872                 "sport": req.sport,
873                 "status": "stored",
874             },
875         )
```

```
876
877
878     def _ingest_remote_segments(
879         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
880     ) -> int:
881         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
882         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
883         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
884         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
885         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
886         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
887         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
888         and prunes any partial rows (destroy-AFTER-confirm)."""
889         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
890         ref = storage.parse_uri(uri)
891         n = 0
892         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
893             local = storage.fetch(
894                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
895             )
896             with open(local, newline="", encoding="utf-8") as f:
897                 for row in csv.DictReader(f):
898                     if n >= max_rows:
899                         logger.warning(
900                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
901                             max_rows,
902                             uri,
903                         )
```

9. user (2026-07-02T03:00:23.370Z)

```
1140         test code that calls ``_execute_processing(req)`` still works.
1141
1142         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
1143         Raises on unexpected internal errors ? the caller records those as a job failure
1144         """
1145         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
1146         # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
1147         if config is not None and not isinstance(config, AppConfig):
1148             # Old signature: _execute_processing(req, progress=None)
1149             if req is None and db is not None and isinstance(db, ProcessRequest):
1150                 req = db
1151                 db = None
1152             elif req is None and isinstance(config, ProcessRequest):
1153                 req = config
1154                 config = None
1155         # Resolve config/db from module globals when not passed explicitly
1156         if config is None:
1157             config = global_config
1158         if db is None:
1159             db = db_instance
1160
1161         notify = progress or (lambda stage: None)
1162
1163         notify("hashing")
```

```
1164        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
1165        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
1166        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
1167        # consumers (hash / validators / segmenter) use the resolved local path.
1168        local_video = storage.acquire_local_input(
1169            req.video_path, getattr(config, "storage", None)
1170        )
1171        video_id = compute_video_id(local_video)
1172        was_reingest = db.get_video(video_id) is not None
1173
1174        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
1175        notify("validating")
1176        logger.info(f"Running validations for {req.video_path}...")
1177        val_results, val_context = registry.run_all_with_context(local_video, config)
1178        failures = [r for r in val_results if not r.passed]
1179
1180        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
1181        default_seg = getattr(
1182            getattr(config, "indexing", None), "default_segmenter", "motion"
1183        )
1184        seg_name = req.segmenter or default_seg
1185        segmenter_actual = None
1186        segmentation_ran = False
1187        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
1188        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
1189        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
1190        compute_actual: Optional[str] = None
1191        response: Optional[Dict[str, Any]] = None
1192        try:
1193            if failures:
1194                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
1195                # status; it no longer destroys the video's prior good segments.
1196                db.add_video(
1197                    video_id,
1198                    req.video_path,
1199                    "failed",
1200                    [r.model_dump() for r in val_results],
1201                )
1202                response = {
1203                    "video_id": video_id,
1204                    "status": "failed",
1205                    "message": "Video failed validation checks.",
1206                    "results": [r.model_dump() for r in val_results],
1207                }
1208                return response
1209
```

10. user (2026-07-02T03:00:33.705Z)

```
1660        except _CompileError as e:
1661            return {"status": "failed", "message": e.detail, "video_id": video_id}
1662
1663        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1664        output_dir = (
1665            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1666        )
1667        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
```

```
1668        # unknown/absent locale keeps the configured default unchanged.
1669        localized_watermark = localize_watermark(stitching.watermark, locale)
1670        compiler = VideoCompiler(
1671            output_dir,
1672            begin_padding=stitching.begin_padding,
1673            end_padding=stitching.end_padding,
1674            watermark=localized_watermark,
1675        )
1676        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1677
1678        notify("stitching")
1679        try:
1680            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1681            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1682            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1683            local_src = storage.acquire_local_input(
1684                video["filepath"], getattr(cfg, "storage", None)
1685            )
1686            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1687        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1688            logger.error(
1689                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1690            )
1691            return {
1692                "status": "failed",
1693                "message": f"Highlight stitching failed: {e}",
1694                "video_id": video_id,
1695            }
1696
1697        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1698        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1699        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1700        notify("finished")
1701        return {
1702            "status": "success",
1703            "filepath": out_path,
1704            "download_url": download_url,
1705            "video_id": video_id,
1706        }
1707
1708
1709    # --- CLI Debug Utility & Orchestration ---
```

11. user (2026-07-02T03:00:34.645Z)

```
2040
2041        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
2042        if args.video_path and not args.server and not args.app:
2043            print(f"[CLI] Processing video file: {args.video_path}")
2044            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
2045            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
2046            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
2047            storage_cfg = getattr(global_config, "storage", None)
2048            try:
2049                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
2050            except ValueError as e:
```

```
2051                print(f"[FAIL] {e}")
2052                return
2053            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
2054                print(f"[FAIL] Video file not found: {args.video_path}")
2055                return
2056            local_video = storage.acquire_local_input(args.video_path, storage_cfg)
2057
2058            video_id = compute_video_id(local_video)
2059            was_reingest = db_instance.get_video(video_id) is not None
2060
2061            # Validate (with-context variant surfaces ffprobe_meta for the report)
2062            print("[CLI] Validating...")
2063            val_results, val_context = registry.run_all_with_context(
2064                local_video, global_config
2065            )
2066            failures = [r for r in val_results if not r.passed]
2067
2068            # Save video status
2069            status = "failed" if failures else "processed"
```

12. user (2026-07-02T03:00:35.663Z)

```
195             return ComputeResult(
196                 returncode=rc,
197                 outputs_uri=outputs_uri,
198                 video_id=video_id,
199                 report=report,
200                 execution=str(execution or ""),
201             )
202
203         # --- storage-backed completion signal (bucket-poll) ---
204         def _read_marker(self, done_uri: str) -> Optional[dict]:
205             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
206             ref = parse_uri(done_uri)
207             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
208                 return None
209             return (
210                 self._read_json(done_uri) or {}
211             )  # empty dict still terminates the poll (present == done)
212
213         def _read_json(self, uri: str) -> dict:
214             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
215             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
216             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
217             try:
218                 ref = parse_uri(uri)
219                 dst = os.path.join(
220                     tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
221                 )
222                 local = fetch(uri, dst, config=self._storage_config)
223                 with open(local, "r", encoding="utf-8") as f:
224                     data: Any = json.load(f)
225                 return data if isinstance(data, dict) else {}
226             except Exception:
227                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
228                 return {}
229
```

13. assistant (2026-07-02T03:00:50.933Z)

All call sites check out. Let me confirm no janitor exists anywhere (tools/cleanup_caches was

mentioned in an error message) and check worker-side patterns.

14. user (2026-07-02T03:00:53.407Z)

```
badminton-highlight-indexer\tools\cleanup_caches.py:6:**purge** (regenerable: frame caches,
staged video copies, handoff clips, temp dirs;
badminton-highlight-indexer\tools\cleanup_caches.py:13:``.partial-<id>`` temp files aborted
uploads leave behind. Reels / DB / CSVs stay durable
badminton-highlight-indexer\tools\cleanup_caches.py:57:        # orphaned `.partial-<id>` temp
files aborted uploads leave behind are both swept
badminton-highlight-indexer\tools\cleanup_caches.py:80:            return ("temp dir", True,
True)
badminton-highlight-indexer\tools\cleanup_caches.py:275:    ``.partial-<id>`` temp files); a
future per-user subdir (PR-3) shows up as an
badminton-highlight-indexer\tools\generate_highlights.py:6:  first, then COPIED into the
destination (atomically: temp name -> os.replace). The
badminton-highlight-indexer\tools\generate_highlights.py:67:    Copy to a temp sibling, then
os.replace (atomic on the same volume). Never deletes
badminton-highlight-indexer\tools\generate_highlights.py:99:        (a pure temporal-preserving
downscale shares the timeline). None = detect+stitch the
badminton-highlight-indexer\tools\generate_highlights.py:389:        "in the same order.
Timestamps transfer 1:1 (pure temporal downscale).",
```

15. user (2026-07-02T03:00:54.471Z)

```
badminton-highlight-indexer\backend\compute\cloud_run.py:220:
tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
```

16. assistant (2026-07-02T03:01:00.416Z)

tools/cleanup_caches.py mentions purging "temp dirs" ? I need to check whether it covers the system-temp `rally-input-*` scratch dirs, since that would weaken the "no janitor exists" claim.

17. user (2026-07-02T03:01:01.434Z)

```
1       """Cache-hygiene + retention sweep for the GPU highlight pipeline.
2
3       Scans the regenerable intermediates the pipeline leaves behind ? in the WSL stage
4       dir (``~/clips``), the Windows ``output/`` tree, and the upload landing zone
5       (``security.upload_dir``, default ``output/uploads``) ? classifies each entry as
6       **purge** (regenerable: frame caches, staged video copies, handoff clips, temp dirs;
7       and, past a retention window, uploaded source videos) or **keep** (durable: trajectory
8       CSVs, the SQLite DB, final reels) and reports the reclaimable space.
9
10      **Retention sweep (T11 ? pre-alpha-exposure):** uploaded source videos in the upload
11      dir are a *privacy*-driven purge, not a *disk*-driven one ? once a tester's footage is
12      older than ``--uploads-retention-days`` (default 7) it is swept, along with the orphaned
13      ``.partial-<id>`` temp files aborted uploads leave behind. Reels / DB / CSVs stay
durable
14      per the cache-hygiene policy (the owner's call, 2026-06-21).
15
16      DRY-RUN BY DEFAULT ? it only *shows* what it would delete. Pass ``--apply`` to delete.
17
18          python -m tools.cleanup_caches                      # report only (safe)
19          python -m tools.cleanup_caches --apply              # delete purge candidates
20          python -m tools.cleanup_caches --keep-video 89605e80ed01   # protect one video's
cache
21          python -m tools.cleanup_caches --older-than 7       # only purge regenerable caches
>7d old
22          python -m tools.cleanup_caches --uploads-retention-days 3   # sweep uploads >3d old
23          python -m tools.cleanup_caches --include-wasbcache  # also drop detector resume
caches
24          python -m tools.cleanup_caches --json               # machine-readable plan (for a
cron/job wrapper)
```

```
25
26      Pairs with the pipeline's own self-cleaning (indexing.{wasb,tracknet}.keep_frames=false,
27      which deletes frames after a SUCCESSFUL run); this tool reclaims what failed/old runs
28      and pre-self-cleaning videos left behind, and enforces the upload retention window
before
29      non-owner (alpha) exposure. Stdlib only; safe to run anywhere (the WSL scan is skipped
30      automatically when ``wsl`` is unavailable). ``--uploads-dir`` must match
``security.upload_dir``
31      (its default matches the config default, ``output/uploads``).
32      """
33
34      from __future__ import annotations
35
36      import argparse
37      import os
38      import shutil
39      import subprocess
40      import sys
41      from dataclasses import dataclass
42      from typing import Dict, List, Optional, Tuple
43
44      # ---- classification (pure; unit-tested) ----------------------------------- #
45
46      # (category, regenerable, default_purge). default_purge=True ? deleted unless a flag
47      # protects it. wasbcache is regenerable but default_purge=False (it enables resume and
48      # is tiny) ? opt in with --include-wasbcache.
49      _KEEP = ("keep", False, False)
50
51
52      def classify(name: str, is_dir: bool, location: str) -> Tuple[str, bool, bool]:
53          """Classify a cache entry by name. Returns (category, regenerable,
default_purge)."""
54          low = name.lower()
55          if location == "uploads":
56              # The upload landing zone (security.upload_dir). Uploaded source videos and the
57              # orphaned `.partial-<id>` temp files aborted uploads leave behind are both
swept
58              # past the retention window (default_purge=True; held by --uploads-retention-
days
59              # in plan_deletions). Anything else here ? e.g. a future per-user subdir (PR-3)
? is
60              # left untouched, since we don't recurse into unknown upload trees.
61              if is_dir:
62                  return ("upload subdir", False, False)
63              if low.startswith(".partial-"):
64                  return ("partial upload", True, True)  # aborted/in-flight chunk file
65              if low.endswith((".mp4", ".avi", ".mov", ".mkv")):
66                  return ("uploaded source", True, True)  # tester footage ? privacy retention
67              return ("other", False, False)
68          if is_dir:
69              if low.endswith("_frames"):
70                  return ("frame cache", True, True)
71              if low.endswith("_wasbcache"):
72                  return (
73                      "detector resume cache",
74                      True,
75                      False,
76                  )  # opt-in via --include-wasbcache
77              if low.startswith("chunks_") and low.endswith("_handoff"):
78                  return ("ai handoff clips", True, True)
79              if low.startswith("tmp"):
80                  return ("temp dir", True, True)
81              return ("other dir", False, False)
82          # files
83          if low.endswith((".csv",)):
```

```
84              return ("trajectory csv", False, False)  # durable analysis output
85          if low.endswith((".db", ".sqlite", ".sqlite3")):
86              return ("database", False, False)  # cached rallies ? never purge
87          if low.endswith((".log", ".sh", ".txt", ".json", ".md")):
88              return ("log/script", False, False)
89          if low.endswith((".mp4", ".avi", ".mov", ".mkv")):
90              # In the WSL stage dir, every video is a regenerable staged copy.
91              if location == "wsl":
92                  return ("staged video", True, True)
93              # In output/, a 1080p/proxy is regenerable; anything else is a produced reel.
94              if "_1080p" in low or "_proxy" in low or low.endswith("_proxy.mp4"):
95                  return ("proxy video", True, True)
96              return ("highlight reel", False, False)  # durable output ? keep
97          return ("other", False, False)
98
99
100     @dataclass
101     class Entry:
102         name: str
103         path: str  # display path (WSL ~/clips/<name> or windows abs)
104         size: int  # bytes
105         mtime: float  # epoch seconds (0 if unknown)
106         is_dir: bool
107         location: str  # "wsl" | "windows" | "uploads"
108
109         @property
110         def category(self) -> Tuple[str, bool, bool]:
111             return classify(self.name, self.is_dir, self.location)
112
113
114     def plan_deletions(
115         entries: List[Entry],
116         *,
117         keep_video: Optional[str],
118         older_than_days: float,
119         include_wasbcache: bool,
120         now: float,
```

18. user (2026-07-02T03:01:29.131Z)

```
T11 follow-up: run the upload retention sweep as an automated scheduled job (hosted alpha)
[OPEN] ## Context

The T11 **upload retention sweep** landed in #299 (merged, `cb7394b`): `tools/cleanup_caches.py`
was extended to purge ? past a retention window ? uploaded source videos and orphaned
`.partial-<id>` files in `security.upload_dir` (default `output/uploads`), alongside the
existing regenerable-cache cleanup. It is **dry-run by default**, keeps reels/DB/CSVs durable,
and emits a machine-r
```

19. user (2026-07-02T03:01:29.288Z)

```
badminton-highlight-indexer\backend\main.py:789:
require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
badminton-highlight-indexer\backend\main.py:1402:
require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
badminton-highlight-indexer\backend\utils\freespace.py:49:def require_free_bytes(
```

20. assistant (2026-07-02T03:01:49.218Z)

```
All five verification points are settled. Calling in the verdict.
```

21. user (2026-07-02T03:02:02.827Z)

```
Structured output provided successfully
```